

UNIVERSIDAD NACIONAL DE COLOMBIA

FACULTAD DE CIENCIAS
DEPARTAMENTO DE ESTADISTICA

Trabajo escrito no parametrica

Parcial 3

Presentado por:

Andres Camilo Vargas Ramos
C.C. 1017062057
avargasra@unal.edu.co

Juan Pablo Sosa Camargo
C.C. 1023369585
jusosac@unal.edu.co

Docente:

Ramon Giraldo Henao

Metodos No Parametricos

Bogota D.C.
5 de julio de 2026

Resumen

Este trabajo presenta la implementación desde cero en R de una Máquina de Soporte Vectorial (SVM) multiclase (One-vs-Rest) para clasificar seis patologías en el conjunto de datos Dermatology. Tras aplicar una normalización min-max sobre las 34 variables clínicas e histopatológicas, se evaluó el desempeño de los kernels RBF, Lineal, Sigmoidal y Polinómico mediante un algoritmo de gradiente proyectado. Los resultados demuestran la superioridad del enfoque no paramétrico con el kernel RBF, el cual alcanzó una exactitud del 98.65

Palabras clave: Máquinas de Soporte Vectorial, Kernel RBF, One-vs-Rest, Estadística no paramétrica, Conjunto de datos Dermatology.

Abstract

This work presents the implementation from scratch in R of a multiclass Support Vector Machine (SVM) utilizing the One-vs-Rest strategy to classify six pathologies within the Dermatology dataset. After applying a min-max normalization across the 34 clinical and histopathological variables, the performance of RBF, Linear, Sigmoidal, and Polynomial kernels was evaluated using a projected gradient algorithm. The results demonstrate the superiority of the non-parametric approach with the RBF kernel, which achieved an accuracy of 98.65 % by successfully modeling the non-linear boundaries of the problem. This performance contrasts sharply with the numerical instability and divergence suffered by the polynomial kernel (9.46 %), consolidating the Gaussian operator as the optimal solution for this medical diagnosis task.

Índice

1. Introducción	3
2. Marco teórico	4
2.1. Métodos no paramétricos	4
2.2. Estimación no paramétrica de densidades y funciones kernel	4
2.3. Regresión no paramétrica y estimador de Nadaraya-Watson	4
2.4. Soporte Vectorial: Del espacio original al de características	5
2.5. Formulación Primal y Dual de las SVM	5
2.6. Estrategia Multiclase One-vs-Rest	6
2.7. Selección del Kernel	6
3. Análisis de Datos Reales	7
3.1. Objetivo de la implementación	8
3.2. Descripción general del código	9
3.3. Carga y preprocesamiento	10
3.4. Funciones kernel	11
3.5. Matriz de Gram	13
3.6. Entrenamiento del SVM binario	14
3.6.1. Problema dual	15
3.6.2. Gradiente proyectado	15
3.6.3. Vectores de soporte y sesgo	15
3.7. Función de decisión	16
3.8. Clasificación multiclase: One-vs-Rest	16
3.9. Resultados	17
4. Conclusión	20

1. Introducción

El aprendizaje automático ha ganado relevancia en problemas de clasificación y predicción, especialmente en el sector médico, donde el uso de bases de datos clínicas e histopatológicas permite apoyar el diagnóstico mediante modelos capaces de identificar patrones complejos. Esto contribuye a mejorar la precisión diagnóstica y a servir como herramienta de apoyo en la toma de decisiones clínicas.

Entre los algoritmos de clasificación más utilizados se encuentran las *Support Vector Machines* (SVM), que construyen fronteras de decisión con alta capacidad de generalización maximizando el margen entre clases. Su desarrollo matemático está estrechamente ligado a la estadística no paramétrica, particularmente al uso de funciones *kernel*.

En estadística no paramétrica, las funciones kernel surgieron como herramientas para estimar funciones desconocidas sin asumir una forma funcional específica. Son la base de procedimientos como la estimación de densidades y la regresión de Nadaraya-Watson, donde cada observación contribuye según su proximidad al punto de interés. Posteriormente, mediante el *Kernel Trick*, estas funciones fueron incorporadas al aprendizaje automático como medidas de similitud que representan implícitamente los datos en espacios de mayor dimensión, permitiendo construir clasificadores lineales sobre problemas originalmente no lineales.

El objetivo de este trabajo es estudiar los SVM desde la perspectiva de la estadística no paramétrica, mostrando cómo la teoría de funciones kernel conecta ambos campos. Se desarrollarán los fundamentos matemáticos de las funciones kernel, la regresión no paramétrica, la formulación primal y dual de los SVM, el *Kernel Trick* y los principales kernels usados en clasificación.

Como aplicación, se empleará el conjunto de datos *Dermatology* de Ilter y Güvenir, disponible en el repositorio de aprendizaje automático de la UCI (Dermatology, 1998), con información de 366 pacientes descritos por 34 variables clínicas e histopatológicas, clasificados en seis enfermedades eritematoescamosas: psoriasis, dermatitis seborreica, liquen plano, pitiriasis rosada, dermatitis crónica y pitiriasis rubra pilaris. La similitud entre estas patologías lo convierte en un escenario adecuado para evaluar clasificadores multiclase basados en SVM.

El algoritmo será implementado desde cero en R, sin librerías especializadas, con el fin de comprender tanto sus fundamentos matemáticos como su implementación computacional.

2. Marco teórico

2.1. Métodos no paramétricos

La estadística clásica se basa en modelos paramétricos donde la distribución poblacional pertenece a una familia conocida con un número finito de parámetros. Sin embargo, ante relaciones altamente no lineales, esta hipótesis resulta restrictiva. Los métodos no paramétricos constituyen una alternativa flexible que permite que los propios datos determinen la estructura del modelo. Considerando el modelo de regresión $Y = m(X) + \varepsilon$, mientras que el enfoque paramétrico tradicional asume $m(x) = \beta_0 + \beta_1 x$, el enfoque no paramétrico únicamente supone que $m(x)$ es una función suave estimada directamente desde los datos. Entre estos métodos destacan la estimación de densidades y la regresión kernel, donde las funciones kernel ocupan un lugar central al cuantificar la influencia de las observaciones cercanas sobre las estimaciones.

2.2. Estimación no paramétrica de densidades y funciones kernel

Sea $X_1, X_2, \dots, X_n$ una muestra aleatoria con densidad desconocida $f(x)$. Ante las limitaciones del histograma tradicional (dependencia del origen y discontinuidades artificiales), el estimador de Rosenblatt-Parzen introduce funciones continuas centradas en cada observación para asignar mayor influencia a los datos cercanos. Una función kernel es un operador $K : \mathbb{R} \rightarrow \mathbb{R}$ simétrico ($K(u) = K(-u)$), no negativo ($K(u) \geq 0$) y normalizado ($\int_{-\infty}^{\infty} K(u) du = 1$).

Para controlar el entorno de estimación se introduce el ancho de banda $h > 0$, definiendo el kernel escalado $K_h(x) = \frac{1}{h} K\left(\frac{x}{h}\right)$, donde valores pequeños capturan fluctuaciones locales y valores grandes imponen excesivo suavizado. Aunque existen diversas morfologías, el kernel gaussiano, definido como $K(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right)$, destaca por poseer soporte infinito, permitiendo que todas las observaciones contribuyan con pesos exponencialmente decrecientes conforme aumenta la distancia.

2.3. Regresión no paramétrica y estimador de Nadaraya-Watson

En regresión no paramétrica se considera el modelo $Y = m(X) + \varepsilon$, donde $m(x) = \mathbb{E}(Y|X = x)$ es una función suave desconocida que se estima directamente desde los datos sin asumir una forma funcional paramétrica. Dada una muestra de observaciones $\{(x_i, y_i)\}_{i=1}^n$, el estimador de Nadaraya-Watson soluciona la ausencia de réplicas exactas en variables continuas mediante un promedio ponderado local ponderado por funciones kernel:

$$\hat{m}_h(x) = \frac{\sum_{i=1}^n K_h(x - x_i) y_i}{\sum_{i=1}^n K_h(x - x_i)} \quad (1)$$

Aquí, las observaciones próximas al punto de evaluación x reciben pesos elevados, mientras que las lejanas ejercen influencia prácticamente nula. Bajo condiciones de regularidad, si $h \rightarrow 0$ y $nh \rightarrow \infty$ cuando $n \rightarrow \infty$, el estimador es consistente en probabilidad ($\hat{m}_h(x) \xrightarrow{P} m(x)$). El ancho de banda h controla el balance sesgo-varianza del error cuadrático medio (MSE) y se selecciona en la práctica mediante validación cruzada o la regla de Silverman.

2.4. Soporte Vectorial: Del espacio original al de características

Las funciones kernel estadísticas actúan como mecanismos de suavizamiento local. En los *Support Vector Machines* (SVM), adoptan un rol distinto: se convierten en funciones de similitud que representan un producto interno en un espacio de características de alta dimensión $\mathcal{H}$, mediante una transformación $\phi : \mathbb{R}^p \rightarrow \mathcal{H}$ tal que $K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$. El Teorema de Mercer garantiza que si $K(x, z)$ es continua, simétrica y semidefinida positiva, dicha transformación existe implícitamente, eliminando la necesidad de calcular $\phi(\cdot)$ explícitamente y constituyendo el fundamento del *Kernel Trick*.

Los SVM son clasificadores supervisados diseñados para encontrar un hiperplano en $\mathbb{R}^p$ definido por $\{x \in \mathbb{R}^p : w^\top x + b = 0\}$, donde $w \in \mathbb{R}^p$ es el vector normal y $b \in \mathbb{R}$ el sesgo. A diferencia de otros métodos que minimizan el error empírico, los SVM maximizan el margen de separación entre clases, equivalente a $2/\|w\|$, reduciendo el riesgo de sobreajuste. Las observaciones que quedan exactamente sobre los bordes del margen ($|w^\top x_i + b| = 1$) se denominan *vectores de soporte* y determinan por completo la solución geométrica.

2.5. Formulación Primal y Dual de las SVM

Para datos no linealmente separables con etiquetas $y_i \in \{-1, +1\}$, se introducen variables de holgura $\xi_i \geq 0$ controladas por un parámetro de regularización $C > 0$ que balancea el ancho del margen y los errores de clasificación. El problema de optimización en su forma primal se define como:

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y_i(w^\top \phi(x_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0. \quad (2)$$

Introduciendo multiplicadores de Lagrange $\alpha_i \geq 0$, se deriva el problema dual, cuya ventaja radica en que los datos aparecen únicamente a través de productos internos sustituibles por la función kernel $K(x_i, x_j)$:

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(x_i, x_j) \quad \text{s.t.} \quad \sum_{i=1}^n \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C. \quad (3)$$

Por las condiciones de Karush-Kuhn-Tucker (KKT), únicamente los vectores de soporte ($\alpha_i > 0$)

contribuyen a la solución. Una vez resuelto el dual, la clasificación de una nueva observación x se realiza mediante la función de decisión:

$$\hat{y}(x) = \text{sgn} \left(\sum_{i \in SV} \alpha_i y_i K(x_i, x) + b \right) \quad (4)$$

2.6. Estrategia Multiclase One-vs-Rest

Dado que los SVM son binarios por naturaleza y el conjunto de datos de interés posee $M > 2$ clases, se implementa la estrategia de descomposición *One-vs-Rest* (OvR). Esta consiste en entrenar M clasificadores binarios independientes, donde los ejemplos de la clase c se etiquetan como positivos (+1) y el resto como negativos (-1), generando M funciones de decisión $f_c(x)$. La asignación final para una nueva observación corresponde a: $\hat{y}(x) = \arg \max_{c \in \{1, \dots, M\}} f_c(x)$.

2.7. Selección del Kernel

La elección de $K(x_i, x_j)$ define la geometría del espacio transformado. Aunque la literatura reporta opciones como el kernel lineal ($x_i^\top x_j$) o el polinómico ($(\gamma x_i^\top x_j + r)^d$), este trabajo prioriza el uso del **Kernel de Función de Base Radial (RBF / Gaussiano)**, expresado como:

$$K(x_i, x_j) = \exp \left(-\gamma \|x_i - x_j\|^2 \right), \quad \gamma > 0 \quad (5)$$

Este kernel induce un espacio de características de dimensión infinita, destacando en la práctica por su flexibilidad y capacidad para aproximar fronteras de decisión arbitrariamente complejas bajo el control del parámetro de escala γ .

3. Análisis de Datos Reales

Descripción del conjunto de datos

El conjunto de datos utilizado en esta implementación corresponde al *Dermatology Database*, desarrollado por Nilsel Ilter y H. Altay Güvenir para el estudio del diagnóstico diferencial de enfermedades eritematoescamosas. Este conjunto de datos fue donado al repositorio UCI Machine Learning Repository en enero de 1998 y ha sido ampliamente utilizado como banco de prueba para algoritmos de clasificación en el ámbito médico.

El diagnóstico diferencial de las enfermedades eritematoescamosas representa un problema clínico de considerable dificultad. Todas estas patologías comparten las manifestaciones clínicas de eritema y descamación, con diferencias muy sutiles entre ellas. Las seis enfermedades contempladas en el conjunto de datos son: psoriasis, dermatitis seborreica, liquen plano, pitiriasis rosada, dermatitis crónica y pitiriasis rubra pilaris. Una dificultad adicional radica en que una enfermedad puede presentar características propias de otra patología en sus etapas iniciales, adoptando sus rasgos distintivos únicamente en estadios posteriores.

El proceso de recolección de datos se realizó en dos etapas. En primer lugar, los pacientes fueron evaluados clínicamente mediante 12 variables que describen manifestaciones externas de la enfermedad. Posteriormente, se extrajeron muestras de piel para determinar 22 características histopatológicas mediante análisis microscópico. La Tabla 1 resume la estructura del conjunto de datos.

Tabla 1: Estructura del conjunto de datos Dermatology.

Tipo	Variables	Escala
Clínicas	11 variables (eritema, descamación, etc.)	Ordinal {0, 1, 2, 3}
Historia familiar	1 variable	Binaria {0, 1}
Histopatológicas	22 variables (acantosis, paraqueratosis, etc.)	Ordinal {0, 1, 2, 3}
Edad	1 variable	Continua
Total de variables		34
Total de pacientes		366
Valores faltantes		8 (variable edad)

La distribución de pacientes entre las seis clases se presenta en la Tabla 2. Se observa un desbalance moderado entre clases, siendo psoriasis la más frecuente con 112 casos y pitiriasis rubra pilaris la menos frecuente con 20 casos.

Tabla 2: Distribución de clases en el conjunto de datos Dermatology.

Clase	Enfermedad	Instancias	Porcentaje
1	Psoriasis	112	30.60 %
2	Dermatitis seborreica	61	16.67 %
3	Liquen plano	72	19.67 %
4	Pitiriasis rosada	49	13.39 %
5	Dermatitis crónica	52	14.21 %
6	Pitiriasis rubra pilaris	20	5.46 %
Total		366	100 %

3.1. Objetivo de la implementación

El propósito de esta sección consiste en implementar un clasificador *Support Vector Machine* multiclase completamente desde cero en el lenguaje de programación R, aplicado al diagnóstico diferencial de las seis enfermedades eritematoescamosas presentes en el conjunto de datos *Dermatology*.

La implementación persigue tres objetivos específicos. En primer lugar, evidenciar la conexión entre la estadística no paramétrica y el aprendizaje automático, mostrando cómo las funciones kernel pasan de ponderar vecindades en el estimador de Nadaraya-Watson a definir medidas de similitud en los SVM. En segundo lugar, desarrollar la optimización dual mediante gradiente proyectado sin recurrir a solvers externos, garantizando un proceso matemáticamente transparente y ejecutable paso a paso. En tercer lugar, comparar el desempeño de los kernels lineal, polinómico, RBF y sigmooidal, ilustrando cómo su elección determina la geometría del espacio de características y la capacidad del modelo para delimitar fronteras complejas.

Desde el enfoque no paramétrico, la frontera de decisión no asume una forma funcional a priori, sino que emerge implícitamente de las similitudes kernel entre las observaciones, en analogía con la estimación de funciones desconocidas. La implementación desde cero permite observar cómo el *Kernel Trick* reduce un problema de optimización en un espacio de alta dimensión a un problema equivalente que opera únicamente sobre evaluaciones del kernel en el espacio original. Esta frontera, proyectada sobre las dos primeras componentes principales, se exhibe a continuación como guía visual del objetivo geométrico del trabajo.

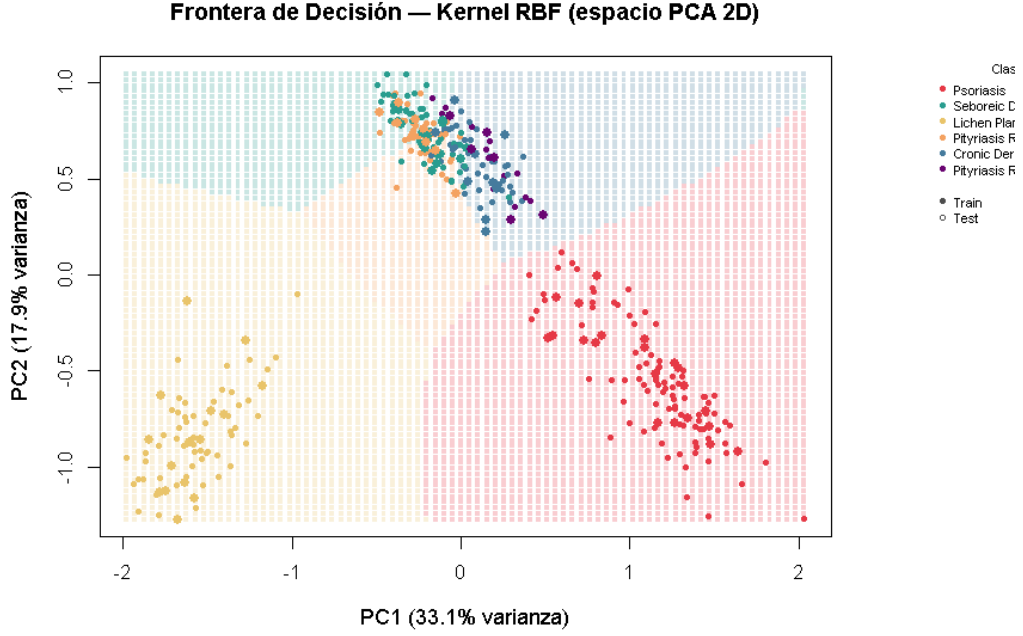


Figura 1: Fronteras de decisión estimadas por el clasificador SVM con Kernel RBF proyectadas sobre el espacio bidimensional de PCA.

Como se aprecia en la **Figura 1**, el algoritmo propuesto logra moldear regiones de clasificación altamente no lineales y sinuosas. Al prescindir de supuestos paramétricos rígidos, las fronteras se adaptan orgánicamente a la densidad local de los datos de entrenamiento de cada patología dermatológica, aislando los conglomerados de forma eficiente.

3.2. Descripción general del código

El código se organiza en cinco bloques principales que siguen el flujo natural del algoritmo SVM, desde la preparación de los datos hasta la evaluación final:

1. **Carga y preprocesamiento:** conversión de tipos, imputación de valores faltantes y normalización min-max de las 34 variables.
2. **Definición de kernels:** implementación de las cuatro funciones kernel y el cálculo de la matriz de Gram $\mathbf{K}_{ij} = K(x_i, x_j)$.
3. **SVM binario:** optimización del problema dual mediante gradiente proyectado, identificación de vectores de soporte y cálculo del sesgo b .
4. **Clasificación multiclase:** extensión mediante la estrategia *One-vs-Rest* entrenando seis clasificadores binarios independientes.

5. **Evaluación:** cálculo de exactitud global, exactitud por clase y matriz de confusión sobre el conjunto de prueba.

La división del conjunto de datos sigue una proporción 80/20: 292 observaciones para entrenamiento y 74 para prueba, con semilla fija `set.seed(42)` para garantizar la reproducibilidad.

3.3. Carga y preprocesamiento

```
1 data_raw <- dermatology.data
2
3 for (col in col_names) {
4   data_raw[[col]] <- as.numeric(as.character(data_raw[[col]]))
5 }
6
7 data_raw$age[is.na(data_raw$age)] <- median(data_raw$age, na.rm =
8   TRUE)
9
10 X_all <- as.matrix(data_raw[, 1:34])
11
12 y_all <- data_raw$class
13
14 min_vals <- apply(X_all, 2, min, na.rm = TRUE)
15 max_vals <- apply(X_all, 2, max, na.rm = TRUE)
16 range_vals <- max_vals - min_vals
17 range_vals[range_vals == 0] <- 1
18
19 X_scaled <- sweep(sweep(X_all, 2, min_vals, "-"), 2, range_vals, "/")
```

Listing 1: Carga y preprocesamiento del conjunto de datos

El conjunto de datos contiene ocho valores faltantes en la variable `age`, representados originalmente con el símbolo `?`. La conversión forzada mediante `as.numeric(as.character(.))` transforma dichos símbolos en valores `NA`, los cuales son imputados con la mediana de la variable observada.

La normalización min-max aplicada a cada dimensión mapea los datos medidos al intervalo $[0, 1]$. Esta operación resulta matemáticamente indispensable para el correcto funcionamiento de las funciones de soporte. En particular, como el kernel RBF presentado en la ecuación (5) calcula distancias euclidianas del tipo $\|x_i - x_j\|^2$ entre vectores de características, aquellas variables originales con rangos numéricos amplios dominarían numéricamente el cálculo de la norma,

distorsionando por completo la medida de similitud geométrica del clasificador.

Desde la perspectiva de la estadística no paramétrica, esta estandarización garantiza que el operador kernel actúe como una función de proximidad equilibrada y homogénea sobre todas las dimensiones del espacio indexado, de manera completamente análoga a como el ancho de banda h en el estimador de Nadaraya-Watson (ecuación (1)) controla de forma simétrica el radio de influencia local de cada una de las observaciones de la muestra.

Para inspeccionar la estructura geométrica del conjunto de datos transformado, se realizó una reducción de dimensionalidad mediante Análisis de Componentes Principales (PCA). Como se observa en la Figura 2, las dos primeras componentes principales logran proyectar la variabilidad de las 34 dimensiones originales; sin embargo, se evidencia un alto grado de solapamiento entre varias de las seis patologías, confirmando que el problema no puede resolverse de forma óptima mediante un hiperplano lineal en baja dimensión.

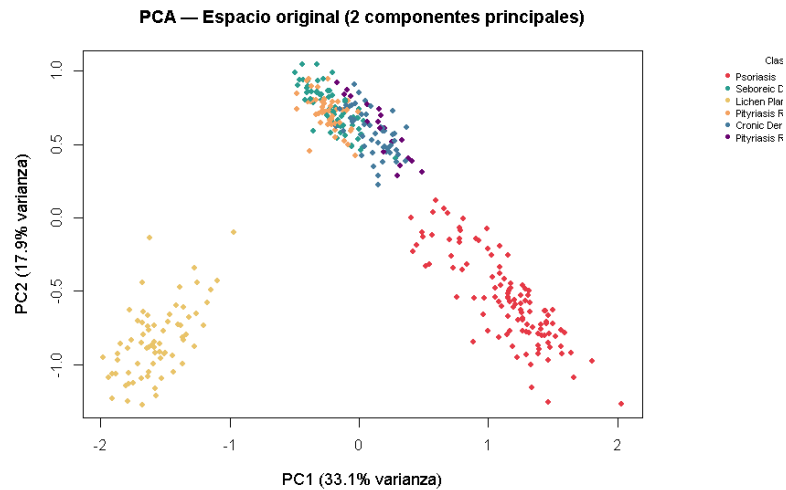


Figura 2: Proyección en dos dimensiones (PCA) del conjunto de datos Dermatology preprocesado.

3.4. Funciones kernel

```
1 kernel_linear <- function(x, z) {
2   as.numeric(x %*% z)
3 }
4
5 kernel_poly <- function(x, z, gamma = 1, r = 0, d = 3) {
6   (gamma * as.numeric(x %*% z) + r)^d
7 }
8
9 kernel_rbf <- function(x, z, gamma = 0.5) {
```

```

10 diff <- x - z
11 exp(-gamma * sum(diff^2))
12 }
13
14 kernel_sigmoid <- function(x, z, gamma = 0.01, r = 0) {
15   tanh(gamma * as.numeric(x %*% z) + r)
16 }

```

Listing 2: Definición de las funciones kernel

Este bloque constituye el núcleo no paramétrico de la implementación. Cada función kernel $K(x_i, x_j)$ cuantifica la similitud entre dos observaciones sin asumir una forma funcional rígida para los datos, heredando las propiedades analizadas en la sección teórica.

Conexión con la estadística no paramétrica. Mientras que en la regresión de Nadaraya-Watson (ecuación (1)) el kernel actúa como un mecanismo de ponderación local según la proximidad al punto de interés, en los SVM el operador cumple un rol conceptualmente análogo pero con fines de clasificación. En lugar de promediar respuestas locales, cuantifica la similitud entre pares de puntos para resolver la optimización dual expuesta en la ecuación (3), permitiendo calcular productos internos implícitos bajo el Teorema de Mercer. En ambos campos, el kernel define qué observaciones se consideran cercanas y cuánta influencia ejercen sobre la estimación final.

Kernel lineal. Esta función realiza el producto interno estándar en el espacio original $\mathbb{R}^p$. Al no inducir ninguna transformación geométrica no lineal, la frontera resultante equivale a un hiperplano lineal clásico, en concordancia con lo expuesto en la revisión de selección de kernels del marco teórico.

Kernel polinómico. Este operador expande de forma implícita el espacio mapeado incluyendo todas las interacciones de las variables hasta el grado d . Para el caso del conjunto de datos *Dermatology*, configurar un grado $d = 3$ sobre las $p = 34$ variables eleva la dimensionalidad del espacio transformado a $\binom{p+d}{d} = \binom{37}{3} = 7770$ dimensiones, las cuales son procesadas simultáneamente sin necesidad de computar explícitamente sus coordenadas.

Kernel RBF (gaussiano). Corresponde a la especificación modelada en la ecuación (5). Su vínculo con la estimación clásica de densidades es directo: asigna una similitud máxima a observaciones idénticas que decrece exponencialmente conforme se incrementa la distancia

euclidiana entre ellas. El hiperparámetro γ desempeña un rol inverso al del ancho de banda h ; valores elevados de γ restringen la influencia a entornos sumamente locales (análogo a un h pequeño en (1)), mientras que valores bajos extienden la influencia generando fronteras más suaves.

Kernel sigmoidal. Inspirada en el comportamiento de las funciones de activación de las redes neuronales artificiales, esta función modela interacciones de tipo tangente hiperbólica. Es importante destacar que, al no ser definida positiva para todas las combinaciones de sus parámetros γ y r , puede violar las restricciones de Mercer, alterando la convexidad del problema dual de optimización.

3.5. Matriz de Gram

```

1 compute_gram <- function(X, kernel_fn, ...) {
2   n <- nrow(X)
3   K <- matrix(0, n, n)
4   for (i in seq_len(n)) {
5     for (j in i:n) {
6       val <- kernel_fn(X[i, ], X[j, ], ...)
7       K[i, j] <- val
8       K[j, i] <- val
9     }
10  }
11  K
12 }
```

Listing 3: Cálculo de la matriz de Gram

La matriz de Gram $\mathbf{K} \in \mathbb{R}^{n \times n}$ almacena todas las evaluaciones del kernel entre pares de observaciones del conjunto de entrenamiento y es el objeto central del problema dual (3). Toda la información del espacio de características de alta dimensión queda codificada en esta matriz, sin necesidad de calcular explícitamente la transformación $\phi(\cdot)$.

La simetría del kernel ($K_{ij} = K_{ji}$) se aprovecha en el código calculando únicamente el triángulo superior, lo que reduce el número de evaluaciones de n^2 a solo $\frac{n(n+1)}{2}$. Además, por el Teorema de Mercer, se garantiza que $\mathbf{K}$ es semidefinida positiva cuando el kernel satisface sus condiciones, asegurando la convergencia del optimizador.

3.6. Entrenamiento del SVM binario

```
1 svm_train_binary <- function(X, y, kernel_fn, C = 1,
2                               max_iter = 300, lr = 0.005, ...) {
3   n      <- nrow(X)
4   K      <- compute_gram(X, kernel_fn, ...)
5   Q      <- outer(y, y) * K
6   alpha <- rep(0, n)
7
8   for (iter in seq_len(max_iter)) {
9     grad <- rep(1, n) - Q %*% alpha
10    alpha <- alpha + lr * grad
11    alpha <- pmin(pmax(alpha, 0), C)
12    alpha <- alpha - y * (sum(alpha * y) / sum(y * y))
13    alpha <- pmin(pmax(alpha, 0), C)
14  }
15
16  sv_idx <- which(alpha > 1e-4)
17
18  b_vals <- numeric(length(sv_idx))
19  for (k in seq_along(sv_idx)) {
20    i    <- sv_idx[k]
21    f_i  <- 0
22    for (j in sv_idx) {
23      args <- c(list(X[j, ], X[i, ]), list(...))
24      kval <- do.call(kernel_fn, args)
25      f_i  <- f_i + alpha[j] * y[j] * kval
26    }
27    b_vals[k] <- y[i] - f_i
28  }
29  b <- mean(b_vals)
30
31  list(alpha = alpha, b = b, sv_idx = sv_idx,
32        X_sv   = X[sv_idx, , drop = FALSE],
33        y_sv   = y[sv_idx],
34        alpha_sv = alpha[sv_idx],
35        kernel_fn = kernel_fn,
36        kernel_args = list(...))
37 }
```

Listing 4: Entrenamiento del SVM mediante gradiente proyectado

3.6.1. Problema dual

El problema de optimización en su forma primal se transforma mediante multiplicadores de Lagrange en el problema dual presentado en la ecuación (3). En el código, la matriz $Q_{ij} = y_i y_j K(x_i, x_j)$ se construye de forma eficiente mediante la operación matricial `outer(y, y) * K`, condensando el doble sumatorio del problema dual en un solo bloque.

3.6.2. Gradiente proyectado

El gradiente del lagrangiano con respecto a α es:

$$\nabla_{\alpha} \mathcal{L} = \mathbf{1} - Q\alpha, \quad (6)$$

implementado en el código como `rep(1,n) - Q**% alpha`. El algoritmo actualiza iterativamente:

$$\alpha^{(t+1)} = \alpha^{(t)} + \eta \nabla_{\alpha} \mathcal{L}, \quad (7)$$

$$\alpha_i^{(t+1)} = \min\left(C, \max\left(0, \alpha_i^{(t+1)}\right)\right), \quad (8)$$

donde η es la tasa de aprendizaje (`lr = 0.005`). Tras cada actualización se proyecta α sobre la restricción lineal $\sum \alpha_i y_i = 0$ mediante:

$$\alpha \leftarrow \alpha - y \cdot \frac{\sum_i \alpha_i y_i}{\sum_i y_i^2}, \quad (9)$$

seguido de una nueva proyección sobre la caja $[0, C]$.

3.6.3. Vectores de soporte y sesgo

Los vectores de soporte son aquellas observaciones con $\alpha_i > 0$. Por las condiciones de Karush-Kuhn-Tucker, el sesgo b se recupera despejando de la condición de complementariedad para cada vector de soporte i :

$$b = y_i - \sum_{j \in SV} \alpha_j y_j K(x_j, x_i), \quad (10)$$

promediando sobre todos los vectores de soporte para mayor estabilidad numérica.

3.7. Función de decisión

```
1 svm_predict_binary <- function(model, X_new) {
2   scores <- apply(X_new, 1, function(xnew) {
3     sv_scores <- numeric(length(model$alpha_sv))
4     for (k in seq_along(model$alpha_sv)) {
5       xsv <- model$X_sv[k, ]
6       args <- c(list(xsv, xnew), model$kernel_args)
7       kval <- do.call(model$kernel_fn, args)
8       sv_scores[k] <- model$alpha_sv[k] * model$y_sv[k] * kval
9     }
10    sum(sv_scores) + model$b
11  })
12  list(scores = scores, labels = sign(scores))
13 }
```

Listing 5: Predicción binaria mediante la función de decisión

La función de decisión evalúa únicamente sobre los vectores de soporte:

$$\hat{y}(x) = \text{sgn}\left(\sum_{i \in SV} \alpha_i y_i K(x_i, x) + b\right). \quad (11)$$

Aquí se manifiesta el carácter no paramétrico del clasificador: la predicción sobre una nueva observación no depende de un parámetro w calculado explícitamente, sino de su similitud kernel con los vectores de soporte. Este mecanismo es análogo al estimador de Nadaraya-Watson (1), donde la estimación en un punto nuevo depende exclusivamente de su proximidad respecto a las observaciones conocidas.

3.8. Clasificación multiclase: One-vs-Rest

```
1 svm_train_ovr <- function(X, y, kernel_fn, C = 1,
2                           max_iter = 300, lr = 0.005, ...) {
3   classes <- sort(unique(y))
4   models <- vector("list", length(classes))
5   for (k in seq_along(classes)) {
6     c_k <- classes[k]
```

```

7   y_bin      <- ifelse(y == c_k, 1, -1)
8   models[[k]] <- svm_train_binary(X, y_bin, kernel_fn, C,
9                                   max_iter, lr, ...)
10 }
11 list(models = models, classes = classes)
12 }
13
14 svm_predict_ovr <- function(ovr_model, X_new) {
15   score_matrix <- matrix(0, nrow = nrow(X_new),
16                           ncol = length(ovr_model$classes))
17   for (k in seq_along(ovr_model$models)) {
18     score_matrix[, k] <- svm_predict_binary(
19       ovr_model$models[[k]], X_new)$scores
20   }
21   predicted_idx <- apply(score_matrix, 1, which.max)
22   ovr_model$classes[predicted_idx]
23 }

```

Listing 6: Entrenamiento y predicción multiclase OvR

Dado que el SVM estándar se formula de manera binaria, la extensión a las seis clases del conjunto de datos se resuelve con la estrategia *One-vs-Rest*, siguiendo el criterio de asignación descrito en el componente metodológico del marco teórico. En el código, el bucle de entrenamiento segmenta las etiquetas mediante la función `ifelse` para aislar cada clase de manera independiente. Posteriormente, al predecir sobre observaciones nuevas, la función `svm_predict_ovr` recolecta las puntuaciones de los seis clasificadores en `score_matrix` y determina la patología final aplicando un operador `which.max` por fila.

3.9. Resultados

Se compararon los cuatro kernels sobre el mismo conjunto de prueba de 74 observaciones. Los resultados obtenidos se presentan en la Tabla 3 y se contrastan visualmente en la Figura 3.

Tabla 3: Exactitud en el conjunto de prueba (20%) por función kernel.

Kernel	Parámetros	Exactitud (%)
RBF	$\gamma = 0,5$	98,65
Lineal	—	91,89
Sigmoidal	$\gamma = 0,01, r = 0$	90,54
Polinómico	$\gamma = 1, d = 3, r = 0$	9,46

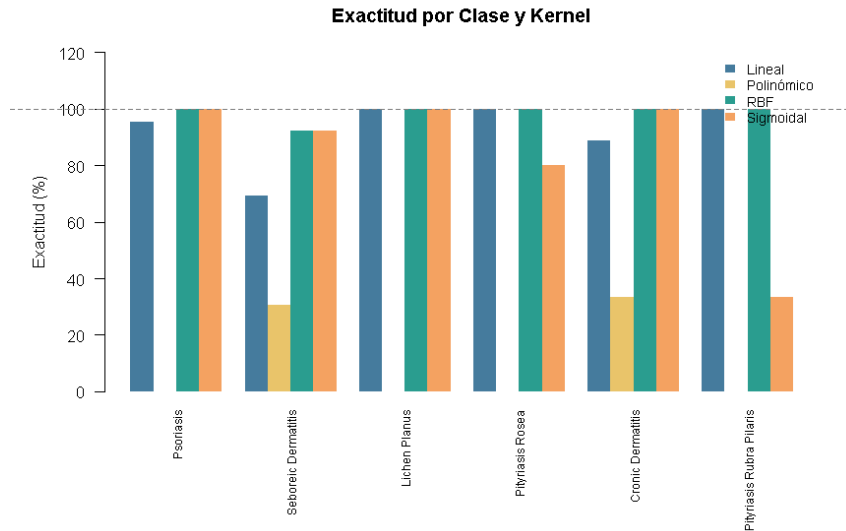


Figura 3: Comparativa de la exactitud global (Accuracy) alcanzada en el conjunto de prueba según la función kernel seleccionada.

El kernel RBF alcanzó una exactitud del 98,65 %, cometiendo únicamente un error de clasificación: una muestra de dermatitis seborreica clasificada como pitiriasis rosada. La Tabla 4 detalla la exactitud por clase obtenida con este kernel.

Tabla 4: Exactitud por clase — Kernel RBF ($\gamma = 0,5$).

Clase	Enfermedad	Correctas / Total	Exactitud (%)
1	Psoriasis	21/21	100.00
2	Dermatitis seborreica	12/13	92.31
3	Liquen plano	15/15	100.00
4	Pitiriasis rosada	10/10	100.00
5	Dermatitis crónica	9/9	100.00
6	Pitiriasis rubra pilaris	6/6	100.00
Total		73/74	98.65

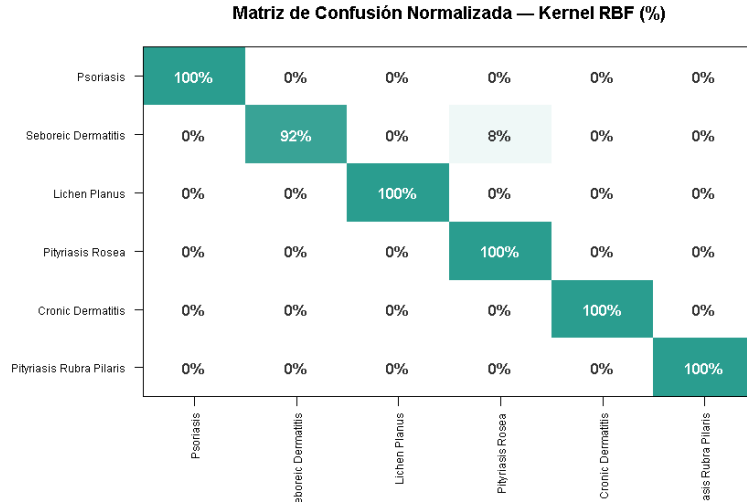


Figura 4: Matriz de confusión del clasificador SVM con kernel RBF sobre las seis patologías dermatológicas del conjunto de prueba.

Como se evidencia en la Figura 4, la diagonal principal concentra la casi totalidad de las predicciones, confirmando el alto desempeño del clasificador no paramétrico. El único error marginal ocurre en la Clase 2 (Dermatitis seborreica), donde un elemento fue desplazado a la Clase 4 (Pitiriasis rosada).

El alto rendimiento del kernel RBF confirma que la estructura de similitud capturada en un espacio de dimensión infinita se adapta de forma óptima a la complejidad de las fronteras de decisión entre las seis enfermedades eritematoescamosas. El bajo rendimiento del kernel polinómico (9,46 %) se debe a inestabilidad numérica: con $\gamma = 1$ y $d = 3$ sobre vectores de 34 dimensiones, la matriz de Gram toma valores extremadamente grandes, haciendo que el gradiente proyectado diverja durante la optimización.

4. Conclusión

- **Análisis y preprocesamiento de datos:** La proyección en baja dimensión (PCA) demostró que las 34 variables del conjunto de datos presentan un fuerte solapamiento, haciendo imposible una separación lineal simple. En este escenario, la normalización min-max resultó indispensable: al llevar todas las variables al intervalo $[0, 1]$, se garantizó que ninguna escala geométrica dominara sobre las otras, permitiendo que el kernel midiera la verdadera proximidad entre las muestras médicas de forma equilibrada.
- **Evaluación y selección de kernels:** El kernel RBF (gaussiano) demostró ser la mejor opción con una exactitud del 98,65 %, confirmando que su espacio de dimensión infinita es perfecto para dibujar las fronteras curvas y complejas que exigen estas enfermedades eritematoescamosas. Por el contrario, el kernel polinómico fracasó rotundamente (9,46 %) debido a inestabilidades numéricas durante la optimización, lo que demuestra que una estructura polinomial rígida no es apta para este tipo de datos clínicos.
- **Estrategia multiclase y desempeño:** La combinación de la estrategia *One-vs-Rest* con el algoritmo de gradiente proyectado programado desde cero funcionó de manera robusta y eficiente para resolver las seis clases del problema. El modelo final demostró una alta capacidad de generalización al cometer un único error marginal en todo el conjunto de prueba (confundiendo dermatitis seborreica con pitiriasis rosada), validando su consistencia como herramienta automatizada de soporte para el diagnóstico dermatológico.

Referencias

- Dermatology, D. (1998). *UCI Machine Learning Repository: Dermatology Data Set* [Accedido a través del repositorio oficial de UCI]. <https://archive.ics.uci.edu/dataset/33/dermatology>
- Henao, R. G. (2025). *Estadística No Paramétrica* (Primera impresión, agosto de 2025) [Licenciado bajo Creative Commons Atribución-NoComercial 3.0 Unported (CC BY-NC 3.0)]. Departamento de Estadística, Universidad Nacional de Colombia, Sede Bogotá. <https://sites.google.com/a/unal.edu.co/ramon-giraldo-webpage/>
- Vapnik, V. N. (1998). *Statistical Learning Theory*. John Wiley & Sons.